The SORA Protocol: A Developer's Guide to Building a Generative Q&A Platform with Next.js, Supabase, and Gemini

Part 1: Foundation & Secure Authentication with NextAuth.js

This foundational part establishes the project's bedrock: a modern Next.js application with a robust, decoupled authentication system. The process involves a meticulous configuration of NextAuth.js to use Supabase as a database backend, a non-trivial setup that requires careful attention to detail for a secure and scalable architecture.

1.1 Project Initialization and Tooling

The first step in any production-grade application is to establish a clean, modern, and maintainable project structure. For Project SORA, this begins with scaffolding a Next.js 14 application using the App Router, which provides a powerful foundation for server-side rendering, static generation, and API handling.

Project Scaffolding

A new Next.js project is initialized using the create-next-app command-line interface. During setup, it is critical to select TypeScript for type safety, ESLint for code quality, and the App Router for access to the latest Next.js features like Server Components and Route Handlers.[1]

Bash

```
npx create-next-app@latest sora-project --typescript --tailwind --eslint --app
cd sora-project
```

This command generates a pre-configured project with Next.js 14, Tailwind CSS, and the App Router, providing an optimal starting point.[1]

Styling and UI Preparation

Tailwind CSS is integrated from the start, following the official Next.js setup guide.[2] The primary configuration files,

tailwind.config.ts and app/globals.css, are the central points for defining the application's design system. To gracefully handle the rendering of AI-generated responses, which are often formatted in Markdown, the @tailwindcss/typography plugin is an essential addition.

Install the typography plugin:

Bash

```
npm install -D @tailwindcss/typography
```

Then, enable it within tailwind.config.ts:

TypeScript

```
import type { Config } from "tailwindcss";

const config: Config = {
  content: [
    "./pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./components/**/*.{js,ts,jsx,tsx,mdx}",
    "./app/**/*.{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {},
  },
```

```
  plugins: [require("@tailwindcss/typography")],
};
export default config;
```

This setup allows for beautiful, readable text blocks by simply applying the prose class to the container rendering the AI's output.[1]

1.2 Architecting Authentication with NextAuth.js v5

Authentication is a critical security component. For Project SORA, NextAuth.js (specifically v5, often referred to as Auth.js) is selected for its flexibility, extensive provider support, and decoupling from any single backend service.

Installation and Core Configuration

The necessary packages for NextAuth.js and its Supabase adapter are installed. Using the @beta tag for next-auth ensures access to the latest v5 features.[4]

```
Bash
```

```
npm install next-auth@beta @auth/supabase-adapter @supabase/supabase-js
```

The authentication logic is centralized in a new root file, auth.ts. This modern pattern in NextAuth.js v5 consolidates the configuration and exports the essential handlers and helper functions (handlers, auth, signIn, signOut) that will be used throughout the application.[4]

A catch-all API route is created at app/api/auth/[...nextauth]/route.ts. This dynamic route is the single endpoint that NextAuth.js uses to manage all authentication flows, including sign-in requests, callbacks from OAuth providers, session management, and sign-out actions.[6] The file's content is minimal, simply re-exporting the handlers from the main configuration file.

app/api/auth/[...nextauth]/route.ts:

```
TypeScript
```

```
export { GET, POST } from "@/auth"
```

1.3 Integrating the Supabase Adapter

The @auth/supabase-adapter allows NextAuth.js to use a Supabase PostgreSQL database for persisting user, account, session, and verification token data. This is a powerful pattern that gives the application full ownership and control over its user data.

Adapter Configuration

Within auth.ts, the SupabaseAdapter is configured. This requires two critical environment variables: SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY. The service role key grants the adapter the necessary administrative privileges to manage the authentication schema programmatically, such as creating users and sessions.[5]

Database Schema Setup

The adapter requires a specific set of tables organized within a dedicated next_auth schema. This separation keeps the authentication data distinct from the application's core business data. The official SQL script provided in the adapter's documentation must be executed in the Supabase SQL Editor to create the users, accounts, sessions, and verification_tokens tables.[5]

auth.ts (Initial Configuration):

TypeScript

```
import NextAuth from "next-auth"
import Google from "next-auth/providers/google"
import { SupabaseAdapter } from "@auth/supabase-adapter"

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    // Google provider will be added in the next step
  ],
  adapter: SupabaseAdapter({
    url: process.env.SUPABASE_URL!,
```

```
    secret: process.env.SUPABASE_SERVICE_ROLE_KEY!,
  }),
  session: {
    strategy: "jwt",
  },
})
```

1.4 Configuring the Google OAuth Provider

To enable social login, a Google OAuth provider is configured. This involves creating credentials in the Google Cloud Platform (GCP) and adding the provider to the NextAuth.js configuration.

GCP OAuth 2.0 Client Setup

1. Navigate to the Google Cloud Platform Console and create a new project.[7]
2. Go to "APIs & Services" > "Credentials".
3. Click "Create Credentials" and select "OAuth client ID".
4. Choose "Web application" as the application type.
5. Under "Authorized JavaScript origins," add your application's base URL (e.g., http://localhost:3000).
6. Under "Authorized redirect URIs," add the NextAuth.js callback URL. This is crucial for Google to securely redirect the user back to the application after authentication. The URL format is /api/auth/callback/google (e.g., http://localhost:3000/api/auth/callback/google).[7]
7. Save the credentials and copy the "Client ID" and "Client Secret". These must be stored securely as environment variables (GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET).

Provider Integration

The GoogleProvider is added to the providers array in auth.ts. The client ID and secret are read from the environment variables.[8] An optional but recommended

signIn callback can be added to enforce additional security checks, such as verifying that the user's email is confirmed and belongs to a specific domain.[8]

auth.ts (with Google Provider):

TypeScript

```
import NextAuth from "next-auth"
import Google from "next-auth/providers/google"
import { SupabaseAdapter } from "@auth/supabase-adapter"

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers:,
  adapter: SupabaseAdapter({
    url: process.env.SUPABASE_URL!,
    secret: process.env.SUPABASE_SERVICE_ROLE_KEY!,
  }),
  session: {
    strategy: "jwt",
  },
  callbacks: {
    async signIn({ account, profile }) {
      if (account?.provider === "google") {
        // Ensure the user has a verified email before allowing sign-in.
        return profile?.email_verified?? false
      }
      return true // Allow other providers
    },
  },
})
```

1.5 Session Management and UI Integration

To make the authentication state available across the application, a session provider is required.

Session Provider Setup

Since the useSession hook is a client-side hook, a new component, AuthProvider, must be created to wrap the SessionProvider.

components/AuthProvider.tsx:

TypeScript

```
'use client';
import { SessionProvider } from 'next-auth/react';

export default function AuthProvider({ children }: { children: React.ReactNode }) {
  return <SessionProvider>{children}</SessionProvider>;
}
```

This AuthProvider is then used to wrap the content of the root layout in app/layout.tsx, making the session context available to all child components.

app/layout.tsx:

TypeScript

```
import AuthProvider from '@/components/AuthProvider';

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        <AuthProvider>{children}</AuthProvider>
      </body>
    </html>
  );
}
```

Login UI

A simple login page at app/login/page.tsx can now be created. This client component will use the signIn function from next-auth/react to initiate the Google OAuth flow when a button is clicked.[9]

app/login/page.tsx:

TypeScript

```
'use client';
import { signIn } from 'next-auth/react';

export default function LoginPage() {
  return (
    <div>
      <button onClick={() => signIn('google', { callbackUrl: '/dashboard' })}>
        Sign in with Google
      </button>
    </div>
  );
}
```

This architecture establishes a secure and flexible authentication system. However, this choice introduces a subtle but critical challenge. By using the @auth/supabase-adapter, the application operates a standalone authentication server that is separate from Supabase's own built-in authentication service (GoTrue).[5] Supabase's powerful Row-Level Security (RLS) relies on a JWT issued by GoTrue, which contains specific claims (like

sub for the user ID) that the database function auth.uid() uses to identify the current user. The JWT generated by NextAuth.js does not contain these claims by default. Consequently, any RLS policy that uses auth.uid() will fail, as the function will return null, effectively breaking the database security model. This necessitates a more advanced configuration, which will be addressed in Part 3, to bridge the gap between the two systems by generating a custom, Supabase-compatible JWT within the NextAuth.js session callback.[5]

Part 2: Architecting the Supabase Database with Declarative Schemas and RLS

This section transitions from UI-driven database management to a professional, code-first methodology. This approach treats the database schema as code, enabling version control, automated migrations, and robust team collaboration. The application's data model will be defined and secured with granular Row-Level Security (RLS) policies.

2.1 Setting up the Supabase CLI and Local Development

A professional workflow requires a local development environment that faithfully mirrors production. The Supabase CLI is the primary tool for achieving this.

CLI Initialization

First, install the Supabase CLI globally or via a package manager. Then, initialize a local Supabase project within the Next.js application's root directory.

Bash

```
supabase init
```

This command creates a supabase directory containing configuration files, a migrations folder for tracking schema changes, and a schemas folder for the declarative schema definition.[11] To connect this local environment to the remote project created in the Supabase dashboard, the

link command is used.

Bash

```
supabase link --project-ref <your-project-ref>
```

This establishes a connection, allowing for schema diffing and deployment.[11]

2.2 Core Schema Design: Conversations and Messages

The data model for Project SORA is centered around two primary entities: conversations and messages.

- **conversations Table:** This table stores metadata for each user's chat session. It includes a user_id column that acts as a foreign key, referencing the id field in the next_auth.users table created by the NextAuth.js adapter. This link is crucial for associating conversations with their rightful owners.
- **messages Table:** This table stores every individual message exchanged within a conversation. It contains the content of the message, a role (either 'user' or 'assistant' to distinguish the speaker), and a conversation_id foreign key that links it back to the parent conversations table.

This normalized structure ensures data integrity and provides a clear, logical organization for the application's data.

2.3 Declarative Schema Management

Instead of applying manual changes through the Supabase Studio UI, a declarative, file-based approach is adopted. This method defines the desired final state of the database schema in .sql files, and the Supabase CLI generates the necessary migration logic to achieve that state.[12]

Schema Definition Files

SQL files are created inside the supabase/schemas directory. The files are executed in lexicographical (alphabetical) order, so a numeric prefix is used to enforce dependency order. The conversations table must be created before the messages table due to the foreign key relationship.[11]

supabase/schemas/01_conversations.sql:

SQL

```
CREATE TABLE public.conversations (
  id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
  user_id UUID REFERENCES next_auth.users(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
  title TEXT
);
```

supabase/schemas/02_messages.sql:

SQL

```
CREATE TABLE public.messages (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  conversation_id UUID REFERENCES public.conversations(id) ON DELETE CASCADE,
  role TEXT NOT NULL CHECK (role IN ('user', 'assistant')),
  content TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);
```

Migration Workflow

The development workflow follows a simple, repeatable pattern:

1. **Modify Schema:** Edit the .sql files in supabase/schemas to reflect the desired database state.[11]
2. **Generate Migration:** Run the db diff command to generate a new migration file. The CLI compares the declarative schema with the current state of the local database and outputs the necessary SQL ALTER statements.[11]
   Bash
   ```
   supabase db diff -f "initial_schema"
   ```
3. **Apply Locally:** Start the local Supabase stack and apply the new migration to verify the changes.[11]
   Bash
   ```
   supabase start
   supabase migration up
   ```
4. **Deploy to Production:** Once verified, push the schema changes to the remote Supabase project.[11]
   Bash
   ```
   supabase db push
   ```

This declarative workflow is a significant advancement over traditional, imperative migration scripts. It allows developers to manage the schema as a single source of truth, simplifying version control and collaboration. Merge conflicts are resolved on the simple CREATE TABLE definitions rather than on complex, order-dependent migration files, making the entire

process more robust and less error-prone.[12]

2.4 Implementing Ironclad Row-Level Security (RLS)

RLS is the cornerstone of Supabase's security model. It allows for the creation of fine-grained authorization rules directly at the database level, ensuring that users can only access data they are permitted to see, regardless of application-level logic.[13]

Enabling RLS

The first and most critical step is to enable RLS on all application tables. By default, this action denies all access (SELECT, INSERT, UPDATE, DELETE) until explicit policies are created to grant permissions.[14]

SQL

```
-- Add these to your schema files or a new migration
ALTER TABLE public.conversations ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.messages ENABLE ROW LEVEL SECURITY;
```

Defining Granular Policies

Specific policies are now crafted for each table and operation, using the auth.uid() function to identify the currently authenticated user.[13]

Policies for the conversations Table:

- **SELECT Policy:** Users can only retrieve conversations that they own.
  SQL
  ```
  CREATE POLICY "Allow users to select their own conversations"
  ON public.conversations FOR SELECT
  TO authenticated
  USING (auth.uid() = user_id);
  ```

- **INSERT Policy:** Users can only create new conversations for themselves. The WITH CHECK clause ensures this rule applies to the new data being inserted.
  SQL

```
CREATE POLICY "Allow users to insert their own conversations"
ON public.conversations FOR INSERT
TO authenticated
WITH CHECK (auth.uid() = user_id);
```

Policies for the messages Table:

Access to messages is more complex, as it depends on the ownership of the parent conversation. This requires a subquery within the policy definition.

- **SELECT Policy:** Users can only view messages that belong to a conversation they own.
  ```
  SQL
  CREATE POLICY "Allow users to select messages in their conversations"
  ON public.messages FOR SELECT
  TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM public.conversations
      WHERE conversations.id = messages.conversation_id AND conversations.user_id =
  auth.uid()
    )
  );
  ```

- **INSERT Policy:** Users can only add messages to conversations they own.
  ```
  SQL
  CREATE POLICY "Allow users to insert messages in their conversations"
  ON public.messages FOR INSERT
  TO authenticated
  WITH CHECK (
    EXISTS (
      SELECT 1 FROM public.conversations
      WHERE conversations.id = messages.conversation_id AND conversations.user_id =
  auth.uid()
    )
  );
  ```

These RLS policies provide a robust "defense in depth" security posture. Even if a bug were present in the application's API layer, the database itself would prevent any unauthorized data access, ensuring user data remains isolated and secure.[13]

Part 3: Building the Backend API and AI Engine

This phase focuses on creating the server-side logic that connects the frontend, the database, and the AI model. All interactions are securely proxied through Next.js Route Handlers, which act as the application's central nervous system.

3.1 Creating Secure API Endpoints with Route Handlers

The modern approach to building APIs in Next.js is through Route Handlers within the App Router.[16] A new file is created at

app/api/chat/route.ts to serve as the primary endpoint for all conversational interactions. This file will export an async function POST(request: NextRequest) to handle incoming chat messages from the frontend client.[18]

3.2 Authenticated Supabase Interaction on the Server

Securely interacting with the database from a server-side context is paramount. This requires a specialized Supabase client and a method for passing the user's authentication context.

Server-Side Supabase Client

The @supabase/ssr package is designed for this purpose, providing helpers to create Supabase clients that can operate in server-side environments like Route Handlers and Server Components.[19] A utility file is created to instantiate this client.

lib/supabase/server.ts:

TypeScript

```
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';
```

```
export const createSupabaseServerClient = (jwt: string) => {
  const cookieStore = cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        get(name: string) {
          return cookieStore.get(name)?.value;
        },
      },
      global: {
        headers: {
          Authorization: `Bearer ${jwt}`,
        },
      },
    }
  );
};
```

Generating and Using a Supabase-Compatible JWT

As established in Part 1, the standard NextAuth.js JWT is incompatible with Supabase RLS. To solve this, a Supabase-compatible JWT must be generated during the authentication process and stored in the session. This is achieved within the callbacks object in auth.ts.

First, install the jsonwebtoken library:

```
Bash
```

```
npm install jsonwebtoken @types/jsonwebtoken
```

Then, update auth.ts to include a jwt and session callback. The session callback signs a new JWT using the SUPABASE_JWT_SECRET, embedding the user's ID from the next_auth.users table into the sub claim, which is what auth.uid() expects. This new token is then attached to the session object.[5]

auth.ts (with JWT generation):

TypeScript

```
//... other imports
import jwt from 'jsonwebtoken';

export const { handlers, auth, signIn, signOut } = NextAuth({
  //... providers and adapter config
  callbacks: {
    async session({ session, token }) {
      const signingSecret = process.env.SUPABASE_JWT_SECRET;
      if (signingSecret) {
        const payload = {
          aud: 'authenticated',
          exp: Math.floor(new Date(session.expires).getTime() / 1000),
          sub: token.sub,
          email: token.email,
          role: 'authenticated',
        };
        session.supabaseAccessToken = jwt.sign(payload, signingSecret);
      }
      return session;
    },
    //... other callbacks
  },
});
```

An accompanying type definition in types/next-auth.d.ts is needed to inform TypeScript about the new property on the session object.[5]

types/next-auth.d.ts:

TypeScript

```
import 'next-auth';

declare module 'next-auth' {
  interface Session {
```

```
    supabaseAccessToken?: string;
  }
}
```

Now, within the app/api/chat/route.ts handler, the user's session is retrieved using the auth() helper. The custom supabaseAccessToken is extracted and used to initialize the server-side Supabase client. This critical step ensures that all subsequent database operations are performed on behalf of the authenticated user and will correctly respect the RLS policies defined in Part 2.[19]

3.3 Integrating the Google Gemini AI Engine

With authentication and database access established, the AI "brain" of the application is integrated.

Gemini SDK Initialization

The official Google GenAI SDK is installed to interact with the Gemini API.[21]

```
Bash
```

```
npm install @google/genai
```

Inside the POST handler, the Gemini client is initialized using the GEMINI_API_KEY from the environment variables. Before calling the model, the full conversation history is fetched from the Supabase database for the current conversation ID. This context is essential for the AI to provide relevant and coherent responses. The generateContent method is then called, passing both the historical messages and the new user prompt.[22]

3.4 Mastering Structured Output with Gemini

A key architectural decision for Project SORA is to leverage Gemini's structured output feature. Instead of receiving a plain text response that requires fragile parsing, the application

can instruct the AI to return a well-defined JSON object.[24]

This is accomplished by defining a responseSchema in the generationConfig passed to the Gemini API. This schema acts as a contract, forcing the model's output to conform to a specific structure. For example, the application could request a response that includes not only the text but also a category and suggested follow-up questions.[24]

app/api/chat/route.ts (simplified logic):

TypeScript

```
import { GoogleGenerativeAI, HarmCategory, HarmBlockThreshold } from '@google/genai';
import { auth } from '@/auth';
import { createSupabaseServerClient } from '@/lib/supabase/server';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);

export async function POST(req: Request) {
  const session = await auth();
  if (!session?.supabaseAccessToken) {
    return new Response('Unauthorized', { status: 401 });
  }

  const { messages, conversationId } = await req.json();
  const supabase = createSupabaseServerClient(session.supabaseAccessToken);

  // 1. Save user message to DB
  await supabase.from('messages').insert({
    conversation_id: conversationId,
    role: 'user',
    content: messages[messages.length - 1].content,
  });

  // 2. Fetch conversation history for context
  //... logic to fetch history from supabase...

  // 3. Call Gemini with structured output requirement
  const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
  const result = await model.generateContent({
    contents: [/* formatted history and new prompt */],
```

```
    generationConfig: {
      responseMimeType: 'application/json',
      responseSchema: {
        type: 'OBJECT',
        properties: {
          response: { type: 'STRING' },
          category: { type: 'STRING' },
          follow_up_suggestions: {
            type: 'ARRAY',
            items: { type: 'STRING' },
          },
        },
        required: ['response'],
      },
    },
  });

  const aiResponse = JSON.parse(result.response.text());

  // 4. Save AI response to DB
  await supabase.from('messages').insert({
    conversation_id: conversationId,
    role: 'assistant',
    content: aiResponse.response,
  });

  // 5. Return structured response to the client
  return new Response(JSON.stringify(aiResponse), {
    headers: { 'Content-Type': 'application/json' },
  });
}
```

This pattern of using a Next.js Route Handler as a secure proxy, combined with Gemini's structured output, effectively transforms the LLM into a reliable microservice. The AI is no longer an unpredictable text generator but a predictable function that returns well-typed data, significantly reducing backend complexity and increasing the robustness of AI-powered features.[24]

Part 4: Crafting the Frontend User Experience

This part of the guide focuses on constructing a polished, responsive, and interactive user interface. It leverages a hybrid approach of Next.js Server Components for secure data loading and Client Components for rich interactivity.

4.1 Building the Dashboard Layout

The main application interface will reside at app/dashboard/page.tsx. This page is designed as a Server Component, which allows for direct and secure data fetching on the server before the page is rendered.

The layout will be a standard dashboard design, featuring a persistent sidebar for navigation (e.g., listing past conversations) and a main content area for the active chat. To accelerate development and ensure a professional aesthetic, pre-built, production-quality components from libraries like TailAdmin or Tailwind UI are highly recommended.[26] These libraries provide responsive, accessible components that can be easily customized.

4.2 Secure Data Fetching in Server Components

One of the primary advantages of the Next.js App Router is the ability to fetch data directly within Server Components. This pattern is both highly performant and secure.

On the main dashboard page (app/dashboard/page.tsx), the auth() helper from auth.ts is used to retrieve the current user's session on the server. If no session exists, the user can be redirected to the login page. With the session object, the custom supabaseAccessToken is extracted and used to initialize the server-side Supabase client. This authenticated client can then securely query the database for user-specific data, such as their list of past conversations.[20]

app/dashboard/page.tsx:

TypeScript

```
import { auth } from '@/auth';
import { redirect } from 'next/navigation';
import { createSupabaseServerClient } from '@/lib/supabase/server';
import ChatInterface from '@/components/ChatInterface';
import ConversationList from '@/components/ConversationList';

export default async function DashboardPage() {
  const session = await auth();

  if (!session ||!session.user ||!session.supabaseAccessToken) {
    redirect('/login');
  }

  const supabase = createSupabaseServerClient(session.supabaseAccessToken);
  const { data: conversations } = await supabase
   .from('conversations')
   .select('id, title')
   .order('created_at', { ascending: false });

  return (
    <div className="flex h-screen">
      <aside className="w-1/4 bg-gray-100 p-4">
        <ConversationList conversations={conversations ||} />
      </aside>
      <main className="w-3/4 flex flex-col">
        <ChatInterface />
      </main>
    </div>
  );
}
```

This approach ensures that sensitive database queries are executed entirely on the server. Only the resulting, non-sensitive data is used to render the HTML that is sent to the client, preventing any exposure of API keys or database credentials and eliminating the possibility of over-fetching data.[30]

4.3 Implementing the Conversational Q&A Interface

The interactive chat functionality is encapsulated within a dedicated Client Component,

<ChatInterface />. This component will manage the real-time state of the conversation.

Leveraging the Vercel AI SDK

To handle the complexities of a streaming chat interface, the Vercel AI SDK is an invaluable tool. It is installed via npm:

Bash

```
npm install ai
```

The SDK provides the useChat hook, a powerful abstraction that manages the entire lifecycle of a chat conversation. It handles the list of messages, the user's input state, the loading/thinking status, and the form submission logic.[32] By default, it makes

POST requests to /api/chat, seamlessly integrating with the backend API created in Part 3.

components/ChatInterface.tsx:

TypeScript

```
'use client';
import { useChat } from 'ai/react';

export default function ChatInterface() {
  const { messages, input, handleInputChange, handleSubmit, isLoading } = useChat({
    api: '/api/chat', // Explicitly define the endpoint
  });

  return (
    <div className="flex flex-col h-full p-4">
      <div className="flex-grow overflow-y-auto mb-4">
        {messages.map((m) => (
          <div key={m.id} className={`p-2 my-1 rounded ${m.role === 'user'? 'bg-blue-100' :
'bg-gray-200'}`}>
            <strong>{m.role === 'user'? 'You: ' : 'SORA: '}</strong>
            {m.content}
          </div>
```

```
      ))}
      {isLoading && <div>Thinking...</div>}
    </div>
    <form onSubmit={handleSubmit}>
      <input
        className="w-full p-2 border rounded"
        value={input}
        onChange={handleInputChange}
        placeholder="Ask SORA anything..."
      />
    </form>
  </div>
 );
}
```

The UI consists of a scrollable area that maps over the messages array provided by the hook to render each message bubble, and a form at the bottom for user input.[34] The

useChat hook elegantly handles streaming responses from the server, automatically updating the messages array as new chunks of data arrive.

This architecture creates a clean separation of concerns. The frontend component is completely decoupled from the specific AI provider being used on the backend. The useChat hook and its corresponding backend helper, StreamingTextResponse, form an abstraction layer. This means that if the project were to switch from Google Gemini to another provider like OpenAI or Anthropic, only the server-side logic in /api/chat/route.ts would need to be modified. The entire frontend, with its complex state management for streaming UI updates, would remain unchanged, making the application highly maintainable and future-proof.[33]

Part 5: Enhancing with Advanced 3D Visuals using react-three-fiber

This section introduces a visually compelling element to the application by integrating high-performance 3D graphics. Using react-three-fiber (R3F), decorative 3D elements can be added to the Next.js application, enhancing the user experience without compromising core functionality or performance.

5.1 Setting up react-three-fiber (R3F) in Next.js

Proper setup is crucial for ensuring that the three.js library and R3F work seamlessly within the Next.js ecosystem.

Installation

The core packages for R3F are installed, including three (the underlying 3D library), @react-three/fiber (the React renderer for Three.js), and @react-three/drei (a vast collection of useful helpers and abstractions).[36]

Bash

```
npm install three @react-three/fiber @react-three/drei
```

Next.js Configuration

A critical configuration step is required in next.config.js to instruct Next.js to correctly process the three library. For modern Next.js versions (13.1+), this is done by adding three to the transpilePackages array.[36]

next.config.mjs:

JavaScript

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  transpilePackages: ['three'],
};

export default nextConfig;
```

This configuration prevents common module resolution errors and ensures smooth integration. For architectural best practices, the react-three-next starter project serves as an excellent reference, particularly for its advanced handling of the 3D canvas within the App Router.[38]

5.2 Creating a Reusable 3D Component

R3F allows developers to build 3D scenes declaratively using familiar React component patterns. A simple, self-contained 3D component is created as a demonstration.

Component Implementation

A new client component, <RotatingIcosahedron />, is created. Inside this component, R3F's JSX syntax is used to define a 3D object. A <mesh> element acts as a container for a geometry (<icosahedronGeometry />) and a material (<meshStandardMaterial />).[39]

To create animation, the useFrame hook is employed. This hook subscribes the component to R3F's render loop, which runs on every screen refresh (typically 60 times per second). The callback function receives the state and delta time, allowing for smooth, frame-rate-independent animations, such as rotating the mesh.[39]

components/canvas/RotatingIcosahedron.tsx:

TypeScript

```
'use client';
import { useRef } from 'react';
import { useFrame } from '@react-three/fiber';
import * as THREE from 'three';

export default function RotatingIcosahedron() {
  const meshRef = useRef<THREE.Mesh>(null!);

  useFrame((state, delta) => {
    if (meshRef.current) {
      meshRef.current.rotation.y += delta * 0.5;
      meshRef.current.rotation.x += delta * 0.2;
    }
  });

  return (
    <mesh ref={meshRef}>
```

```
    <icosahedronGeometry args={} />
    <meshStandardMaterial color="royalblue" wireframe />
   </mesh>
  );
}
```

5.3 Integrating the 3D Element into the Dashboard

To render any R3F component, it must be placed inside a <Canvas> component. The <Canvas> component is the root of the 3D scene, responsible for setting up the WebGL renderer and camera.

This <Canvas> can be placed anywhere within the application's component tree. For Project SORA, it can be added to the dashboard page as a decorative background element, positioned and sized using standard Tailwind CSS classes. To make the 3D object visible, basic lighting elements like <ambientLight /> and <pointLight /> are added as children of the <Canvas>.[39]

app/dashboard/page.tsx (with 3D element):

TypeScript

```
//... other imports
import { Canvas } from '@react-three/fiber';
import RotatingIcosahedron from '@/components/canvas/RotatingIcosahedron';

export default async function DashboardPage() {
  //... data fetching logic...

  return (
    <div className="relative flex h-screen">
     {/* 3D Canvas as a background element */}
     <div className="absolute inset-0 z-0">
       <Canvas>
         <ambientLight intensity={0.5} />
         <pointLight position={} />
         <RotatingIcosahedron />
```

```
      </Canvas>
    </div>

    {/* Main UI on top */}
    <div className="relative z-10 flex w-full">
      <aside className="w-1/4 bg-gray-800 bg-opacity-50 p-4 text-white">
        {/*... ConversationList... */}
      </aside>
      <main className="w-3/4 flex flex-col bg-white bg-opacity-75">
        {/*... ChatInterface... */}
      </main>
    </div>
  </div>
 );
}
```

5.4 Performance and Optimization

While 3D graphics can be resource-intensive, R3F and its ecosystem provide numerous tools for optimization. By isolating 3D logic into specific, self-contained components and leveraging helpers from @react-three/drei, impressive visuals can be added without degrading the core application's performance.[38] For instance,

<OrbitControls> can be added for user interaction, and <PerformanceMonitor> can dynamically adjust quality based on the user's device capabilities.

A more advanced architectural pattern, demonstrated by the react-three-next starter, involves using a single, persistent <Canvas> in the root layout.[38] In a standard R3F implementation, navigating away from a page unmounts its

<Canvas>, destroying the entire WebGL context and all its assets. This can cause a jarring flash and performance hit when navigating back. The react-three-next approach uses a library called tunnel-rat to "portal" 3D components from different pages into the one persistent canvas. This means the WebGL context is never destroyed during navigation, enabling seamless, app-like transitions between pages with 3D content and preserving the 3D scene's state. This transforms the user experience from a series of disconnected 3D scenes into a single, persistent 3D world.

Part 6: Production Deployment on Vercel

The final stage of the project lifecycle is deploying the application to a production environment. Vercel, the platform created by the developers of Next.js, provides a seamless and highly optimized hosting solution. The primary focus of this section is on the correct and secure configuration of environment variables, a common point of failure in deployments.

6.1 Preparing for Deployment

The deployment process begins with version control. The application's code must be pushed to a Git repository hosted on a platform like GitHub, GitLab, or Bitbucket.

Once the repository is ready, a new project is created on the Vercel dashboard. Vercel's Git integration allows for the direct import of the repository. Vercel will automatically detect that it is a Next.js project and configure the optimal build settings and deployment pipeline.[41]

6.2 Mastering Environment Variable Management

Environment variables are used to store sensitive information and configuration details that vary between environments (development, preview, production). Correctly managing these variables is critical for both functionality and security.[42]

Vercel provides a settings UI for managing environment variables. It is essential to add all the secrets required by the application, such as API keys and database credentials, to this UI.[41] Vercel distinguishes between Production, Preview, and Development environments, allowing for different variable values in each context.[43]

A crucial security consideration is the distinction between server-side and client-side variables. In Next.js, any variable prefixed with NEXT_PUBLIC_ will be embedded in the client-side JavaScript bundle, making it publicly accessible. Therefore, sensitive secrets must **never** use this prefix.[30]

To ensure a successful deployment, the following checklist consolidates all required environment variables for Project SORA.

Table 1: Comprehensive Environment Variable Checklist

Variable Name	Description	Source	Scope (Local, Dev, Preview, Prod)	Public?
AUTH_SECRET	Secret key for signing NextAuth.js JWTs.	Generate with openssl rand -base64 32 [44]	All	No
AUTH_URL	Canonical URL of the deployment.	Vercel (auto-set) / http://localhost :3000 [45]	Local Only	No
GOOGLE_CLIE NT_ID	OAuth Client ID for Google Login.	Google Cloud Console [7]	All	No
GOOGLE_CLIE NT_SECRET	OAuth Client Secret for Google Login.	Google Cloud Console [7]	All	No
NEXT_PUBLIC_ SUPABASE_UR L	Public URL for your Supabase project.	Supabase Dashboard > API Settings [41]	All	Yes
NEXT_PUBLIC_ SUPABASE_AN ON_KEY	Public anonymous key for Supabase.	Supabase Dashboard > API Settings [41]	All	Yes
SUPABASE_SE RVICE_ROLE_K EY	Secret service key for admin-level access.	Supabase Dashboard > API Settings [5]	All (Server-side only)	No
SUPABASE_JW	Secret for	Supabase	All	No

T_SECRET	signing custom JWTs for RLS.	Dashboard > API Settings [46]	(Server-side only)	
GEMINI_API_KEY	API key for accessing the Google Gemini API.	Google AI Studio [23]	All (Server-side only)	No

While Vercel's official Supabase integration can automate the syncing of Supabase-related variables, this does not cover secrets from other services like Google Cloud, Gemini, or NextAuth.js.[47] Therefore, a manual review against this checklist is a vital step to ensure all required secrets are present and correctly configured before deployment.

6.3 The Deployment Process

With the repository linked and environment variables set, a production deployment is triggered by pushing a commit to the project's main branch (e.g., main or master).

Vercel's dashboard provides real-time logs for the build and deployment process. These logs are the first place to look for errors. Common issues, such as missing environment variables, will often cause the build to fail with a descriptive error message.[42] If the default build settings inferred by Vercel fail, it may be necessary to manually override the build command or root directory in the project settings.[42]

Upon successful deployment, Vercel assigns a unique URL to the application, making Project SORA live and accessible to the world.

Conclusion

This guide has detailed the architectural decisions and implementation steps required to build Project SORA, a sophisticated, AI-powered Q&A platform. The architecture deliberately integrates a suite of modern, powerful technologies, each chosen for a specific purpose. Next.js with the App Router provides a high-performance foundation for both server-rendered pages and secure API endpoints. NextAuth.js, paired with the Supabase adapter, offers a flexible and decoupled authentication system, giving the application full control over user data. Supabase itself serves as a robust PostgreSQL backend, with its security model

fundamentally hardened by code-first, declarative schemas and ironclad Row-Level Security policies. The intelligence layer is powered by Google Gemini, transformed from a simple text generator into a reliable data service through the use of structured output. The frontend experience is made dynamic and responsive with the Vercel AI SDK and enhanced with visually stunning, performant 3D elements via react-three-fiber. Finally, Vercel provides a seamless, zero-configuration deployment pipeline.

The key architectural patterns—such as generating a custom Supabase JWT within the NextAuth.js session to enable RLS, using Route Handlers as a secure AI proxy, and leveraging the abstraction of the useChat hook—are not merely implementation details. They represent a series of strategic choices designed to create a secure, scalable, and maintainable application. By following this protocol, developers can confidently construct complex, production-grade applications that stand at the forefront of modern web development.

Works cited

1. Next.js 14 Tutorial: Create a Blog with App Router and Tailwind CSS - Djamware, accessed on September 22, 2025, https://www.djamware.com/post/68ba5b20b9ea742423544d8f/nextjs-14-tutorial-create-a-blog-with-app-router-and-tailwind-css
2. Install Tailwind CSS with Next.js - Tailwind CSS, accessed on September 22, 2025, https://tailwindcss.com/docs/guides/nextjs
3. Getting Started: CSS | Next.js, accessed on September 22, 2025, https://nextjs.org/docs/app/getting-started/css
4. Migrate to NextAuth.js v5, accessed on September 22, 2025, https://authjs.dev/getting-started/migrating-to-v5
5. Supabase - Auth.js, accessed on September 22, 2025, https://authjs.dev/getting-started/adapters/supabase
6. Next.js authentication with Supabase and NextAuth: A Deep Dive (Part 1) - Medium, accessed on September 22, 2025, https://medium.com/@sidharrthnix/next-js-authentication-with-supabase-and-nextauth-js-part-1-of-3-76dc97d3a345
7. Login with Google | Supabase Docs, accessed on September 22, 2025, https://supabase.com/docs/guides/auth/social-login/auth-google
8. Google | NextAuth.js, accessed on September 22, 2025, https://next-auth.js.org/providers/google
9. Effortless Authentication: Prisma, Supabase & Google OAuth in Next.js | by Yoonji | Medium, accessed on September 22, 2025, https://medium.com/@navynj/effortless-authentication-prisma-supabase-google-oauth-in-next-js-564ba587881b
10. NextAuth.js Supabase Adapter - YouTube, accessed on September 22, 2025, https://www.youtube.com/watch?v=EdYQ9fF-hz4
11. Declarative database schemas | Supabase Docs, accessed on September 22, 2025, https://supabase.com/docs/guides/local-development/declarative-database-sche

mas
12. Supabase Declarative Schema: A Developer's Guide - Emilio Taylor, accessed on September 22, 2025, https://emiliotaylor.medium.com/supabase-declarative-schema-a-developers-guide-2d261706ddb0
13. Mastering Supabase RLS - "Row Level Security" as a Beginner ..., accessed on September 22, 2025, https://dev.to/asheeshh/mastering-supabase-rls-row-level-security-as-a-beginner-5175
14. Row Level Security | Supabase Docs, accessed on September 22, 2025, https://supabase.com/docs/guides/database/postgres/row-level-security
15. Securing your API | Supabase Docs, accessed on September 22, 2025, https://supabase.com/docs/guides/api/securing-your-api
16. Building APIs with Next.js, accessed on September 22, 2025, https://nextjs.org/blog/building-apis-with-nextjs
17. File-system conventions: route.js | Next.js, accessed on September 22, 2025, https://nextjs.org/docs/app/building-your-application/routing/route-handlers
18. File-system conventions: route.js | Next.js, accessed on September 22, 2025, https://nextjs.org/docs/app/api-reference/file-conventions/route
19. Setting Up Supabase in Next.js: A Comprehensive Guide | by Yagyaraj - Medium, accessed on September 22, 2025, https://medium.com/@yagyaraj234/setting-up-supabase-in-next-js-a-comprehensive-guide-78fc6d0d738c
20. Setting up Server-Side Auth for Next.js | Supabase Docs, accessed on September 22, 2025, https://supabase.com/docs/guides/auth/server-side/nextjs
21. Gemini API libraries | Google AI for Developers, accessed on September 22, 2025, https://ai.google.dev/gemini-api/docs/libraries
22. Text generation | Gemini API | Google AI for Developers, accessed on September 22, 2025, https://ai.google.dev/gemini-api/docs/text-generation
23. Gemini API quickstart | Google AI for Developers, accessed on September 22, 2025, https://ai.google.dev/gemini-api/docs/get-started/node
24. Structured output | Gemini API | Google AI for Developers, accessed on September 22, 2025, https://ai.google.dev/gemini-api/docs/structured-output
25. Generate structured output (like JSON and enums) using the Gemini API | Firebase AI Logic, accessed on September 22, 2025, https://firebase.google.com/docs/ai-logic/generate-structured-output
26. TailAdmin: Free Tailwind CSS Admin Dashboard Template, accessed on September 22, 2025, https://tailadmin.com/
27. Stacked Layouts - Application UI - Tailwind CSS, accessed on September 22, 2025, https://tailwindcss.com/plus/ui-blocks/application-ui/application-shells/stacked
28. Tailwind Plus - Official Tailwind UI Components & Templates, accessed on September 22, 2025, https://tailwindcss.com/plus
29. Build a User Management App with Next.js | Supabase Docs, accessed on September 22, 2025,

https://supabase.com/docs/guides/getting-started/tutorials/with-nextjs
30. Next.js Security Best Practices - Makerkit, accessed on September 22, 2025, https://makerkit.dev/docs/next-supabase-turbo/security/nextjs-best-practices
31. Fetching Data - App Router - Next.js, accessed on September 22, 2025, https://nextjs.org/learn/dashboard-app/fetching-data
32. NextJS Chatbot with Gemini API to generate textual adventure games - GitHub, accessed on September 22, 2025, https://github.com/rukshar69/nextjs-gemini-chatbot
33. Getting Started: Next.js App Router - AI SDK, accessed on September 22, 2025, https://ai-sdk.dev/docs/getting-started/nextjs-app-router
34. Build an AI Chatbot with Next.js and OpenAI API: Full Step-by-Step ..., accessed on September 22, 2025, https://www.codesmith.io/blog/building-an-ai-powered-chatbot-with-next-js-and-openai
35. Building Your First AI Chatbot with Next.js and OpenAI - AI Development Blog, accessed on September 22, 2025, https://www.howtobuild.app/blog/building-ai-chatbot-nextjs-openai
36. Installation - React Three Fiber, accessed on September 22, 2025, https://r3f.docs.pmnd.rs/getting-started/installation
37. pmndrs/react-three-fiber: A React renderer for Three.js - GitHub, accessed on September 22, 2025, https://github.com/pmndrs/react-three-fiber
38. pmndrs/react-three-next: React Three Fiber, Threejs, Nextjs ... - GitHub, accessed on September 22, 2025, https://github.com/pmndrs/react-three-next
39. Introduction - React Three Fiber, accessed on September 22, 2025, https://r3f.docs.pmnd.rs/getting-started/introduction
40. React Three Fiber: Introduction, accessed on September 22, 2025, https://r3f.docs.pmnd.rs/
41. How to Build a Next.js App with Supabase & Shadcn — and Deploy It on Vercel - Medium, accessed on September 22, 2025, https://medium.com/@ronaldmaslog13/how-to-build-a-next-js-app-with-supabase-shadcn-and-deploy-it-on-vercel-73798d3f16c6
42. Deploy Next.js Supabase to Vercel - Makerkit, accessed on September 22, 2025, https://makerkit.dev/docs/next-supabase-turbo/going-to-production/vercel
43. Environment variables - Vercel, accessed on September 22, 2025, https://vercel.com/docs/environment-variables
44. Options | NextAuth.js, accessed on September 22, 2025, https://next-auth.js.org/configuration/options
45. Deployment - NextAuth.js, accessed on September 22, 2025, https://next-auth.js.org/deployment
46. Build a task manager with Next.js, Supabase, and Clerk, accessed on September 22, 2025, https://clerk.com/blog/nextjs-supabase-clerk
47. Supabase for Vercel, accessed on September 22, 2025, https://vercel.com/marketplace/supabase